

Cifrados recomendados

Protocolos y cifrados seguros, aceptables y obsoletos
para implementaciones criptográficas modernas

Parte I · Resumen ejecutivo
Parte II · Guía detallada

hackingyseguridad.com · Octubre de 2026

Fuente y descarga
https://github.com/hackingyseguridad/cifrados/

Contenido

Parte I · Resumen ejecutivo 3

1.- Protocolos y cifrados recomendados 3

2.- Cifrados y protocolos no recomendados (Obsoletos rotos o vulnerables) 4

3.- Certificado digital, recomendaciones de seguridad 4

4.- Escaneo subrutinas 4

Parte II · Guía detallada 5

Introducción 5

Niveles de seguridad 5

Protocolos recomendados 5

Cifrados y protocolos no recomendados 9

Certificados digitales 11

Herramientas de Escaneo 11

Guía de Instalación 12

Casos de Uso 13

Resumen 14

Estructura del repositorio 15

Referencias 15

Disclaimer 15

Fuente original y descarga 16

Parte I · Resumen ejecutivo

Tablas de referencia rápida: qué usar en seguridad media y alta, qué evitar y cómo escanear. Fuente: README.md, del repositorio hackingyseguridad/cifrados.

1.- Protocolos y cifrados recomendados

Protocolo	Seguridad Media (Recomendado)	Seguridad Alta (Recomendado)
Protocolos TLS	TLS 1.2 (solo ECDHE + AEAD), TLS 1.3	TLS 1.3
Cifrado de transporte (TLS/SSH)	AES-128-GCM, CHACHA20-POLY1305	AES-256-GCM, CHACHA20-POLY1305
Intercambio de claves (TLS/IPsec)	ECDHE (X25519, P-256, P-384), DH grupo 14 (2048 bits)	ECDHE (P-384, P-521, X25519), DH grupo 21 (curva P-521)
Funciones hash (HMAC)	SHA-2 (256, 384 bits)	SHA-2 (512 bits), SHA-3
Firma digital	ECDSA (P-256), RSA (≥3072 bits)	EdDSA (Ed25519), ECDSA (P-384), RSA (4096 bits)
IPsec IKEv2	AES-256-GCM, SHA-2, DH grupo 19 (curva P-256)	AES-256-GCM, SHA-384/512, DH grupo 20 (P-384) o 21 (P-521) + ML-KEM (RFC 9370, híbrido)
SSH v2	curve25519-sha256, aes256-gcm@openssh.com o aes256-ctr, hmac-sha2-256-etm@openssh.com (OpenSSH ≥ 10.6 recomendado)	mlkem768x25519-sha256, ssh-ed25519, aes256-gcm@openssh.com, chacha20-poly1305@openssh.com (OpenSSH ≥ 10.6 recomendado)
WiFi	WPA2-AES/CCMP (con PMF 802.11w), WPA3-SAE	WPA3-Enterprise modo 192 bits (GCMP-256, P-384, SHA-384)
PGP/GPG	AES-192, ECDH (Curve25519), ECDSA	AES-256, EdDSA (Ed25519), RSA 4096, OpenPGP v6 (RFC 9580)
Post-cuántico (PQC)	X25519MLKEM768 (híbrido), ML-KEM-768 (Kyber), ML-DSA-65 (Dilithium)	ML-KEM-1024 (híbrido con ECDH P-384), ML-DSA-87, SLH-DSA (SPHINCS+), FN-DSA (Falcon), LMS/XMSS (firmware)
Contraseñas / KDF	PBKDF2-HMAC-SHA256 (≥600.000 iteraciones), bcrypt (coste ≥12)	Argon2id, scrypt
Cifrado en reposo	AES-256-XTS (disco), AES-256-GCM	AES-256-GCM-SIV, XChaCha20-Poly1305 (nonce nunca repetido)
Cifrado asimétrico	RSA-OAEP (≥3072 bits)	HPKE (RFC 9180) con X25519 o P-384

2.- Cifrados y protocolos no recomendados (Obsoletos rotos o vulnerables)

Tipo / Protocolo	algoritmos / cifrados a evitar (Inseguros)
Protocolos SSL/TLS obsoletos	SSLv2, SSLv3, TLS 1.0, TLS 1.1
Cifrados de bloque débiles	DES (40/56 bits), 3DES, RC2, RC4, IDEA, CAST5, Blowfish, suites NULL / EXPORT / anónimas (aNULL, eNULL)
Modos de operación inseguros	AES-ECB, AES-CBC (sin Encrypt-then-MAC), AES-CFB, AES-CTR sin MAC (cualquier modo sin autenticación)
Intercambio de clave inseguro	RSA estático (sin PFS; ROBOT, Marvin), DH/ECDH estático, PSK sin (EC)DHE, grupos DH < 2048 bits, curvas < 224 bits
Funciones hash rotas	MD2, MD4, MD5, SHA-1 (en firmas y certificados; HMAC-SHA1 es solo obsoleto)
Firma digital débil	RSA < 2048 bits, DSA, ECDSA con curvas < 224 bits o no estándar, ssh-rsa (SHA-1; usar rsa-sha2-256/512), RSASHA1 (DNSSEC)
IPsec	IKEv1 (modo agresivo), grupos DH 1, 2 y 5, 3DES, MD5/SHA-1
SSH	SSH v1; cifrados: 3des-cbc, arcfour, blowfish-cbc, aes*-cbc; MACs: hmac-md5, hmac-sha1 (obsoletos); claves ssh-rsa (SHA-1), ssh-dss; OpenSSH < 9.6 (Terrapin)
WiFi	WEP, WPA-TKIP, WPA2 con TKIP, WPS activado, WPA3 en modo transición o sin PMF
PGP/GPG	RSA < 3072 bits, DSA, ElGamal, IDEA, CAST5, 3DES, SHA-1, mensajes sin MDC/AEAD (EFAIL)
Almacenamiento	MD5, SHA-1, SHA-256 sin sal/KDF para contraseñas, DES, 3DES, ECB, AES-128 (en contextos de alta seguridad)
No rotos, pero no recomendados (compatibilidad)	CAMELLIA, ARIA, Twofish, Serpent: válidos, pero poco soportados; preferir AES o ChaCha20

Vulnerabilidades conocidas asociadas: POODLE, BEAST, CRIME, Lucky13, Sweet32 (3DES), KRACK (WPA2), Dragonblood (WPA3), FragAttacks (WiFi), Logjam, FREAK, DROWN, SLOTH, Heartbleed, ROBOT y Marvin (RSA PKCS#1 v1.5), Raccoon (DH), Terrapin (SSH), EFAIL (PGP), BlastRADIUS (RADIUS/MD5), SHAttered (SHA-1), ataques a CBC (padding oracle, bit flipping)

+ cifrados recomendados

+ cifrados recomendados 2

3.- Certificado digital, recomendaciones de seguridad

https://github.com/hackingyseguridad/certificado/

4.- Escaneo subrutinas

testssl.sh IP/rango

sh scan_cifrados.sh lista de IP en ip.txt

nmap --script ssl-enum-ciphers -p 443 IP

ssh-audit IP

http://www.hackingyseguridad.com/

Parte II · Guía detallada

Desarrollo completo de cada categoría, con tablas comparativas, configuraciones de ejemplo, casos de uso y referencias. Fuente: LEEME.md. Leyenda: ● recomendado / alta · ● aceptable / media · ● no usar / crítico.

Introducción

Este repositorio contiene una guía completa sobre **criptografía moderna**, cifrados seguros y protocolos de seguridad recomendados

Objetivo

Proporcionar recomendaciones actualizadas sobre qué cifrados usar, cuáles evitar y por qué, considerando tanto la seguridad a corto plazo como la resistencia post-cuántica.

Niveles de seguridad

Se clasifica las recomendaciones en **4 niveles de seguridad**:

Nivel	Descripción	Casos de Uso	Archivos
Seguridad Baja	Cifrados legacy, no recomendados para datos sensibles	Sistemas heredados, compatibilidad	`seguridad_baja.txt`
Seguridad Media	Aceptables para la mayoría de aplicaciones comerciales	E-commerce, aplicaciones web, HTTPS	`seguridad_media.txt`
Seguridad Alta	Recomendado para datos muy sensibles y financieros	Banca, datos personales, HIPAA	`seguridad_alta.txt`
Seguridad Militar	Nivel máximo, usado en contextos de defensa (referencia: CNSA 2.0)	Secretos de estado, datos clasificados	`seguridad_militar.txt`

Protocolos recomendados

Tabla Comparativa - Protocolos TLS/SSL

Protocolo	Estado	Año	Recomendación	Notas
SSLv2	✗ Roto	1995	✗ NO USAR	Completamente comprometido (DROWN)
SSLv3	✗ Roto	1996	✗ NO USAR	Vulnerable a POODLE
TLS 1.0	✗ Deprecado	1999	✗ NO USAR	Deprecado por RFC 8996; vulnerable a BEAST
TLS 1.1	✗ Deprecado	2006	✗ NO USAR	Deprecado por RFC 8996; no soporta AEAD, POODLE
TLS 1.2	✓ Soportado	2008	● ACEPTABLE	Solo con ECDHE + AEAD (GCM, ChaCha20); sin CBC ni RSA estático
TLS 1.3	✓ Actual	2018	● RECOMENDADO	Estado del arte, eliminó debilidades

Nota: TLS 1.4 no existe. El soporte post-cuántico llega a TLS 1.3 mediante grupos de intercambio híbridos (X25519MLKEM768).

Cifrados simétricos recomendados

Cifrado	Tamaño clave	Modo	Seguridad	Comentario
AES-128	128 bits	GCM	● Media	Aceptable, estándar NIST
AES-192	192 bits	GCM	● Alta	Buen balance; no existe en suites TLS (sí en IPsec, PGP y almacenamiento)
AES-256	256 bits	GCM	● Muy Alta	Máxima seguridad (recomendado)
ChaCha20-Poly1305	256 bits	AEAD	● Muy Alta	Rápido en CPU sin AES-NI
AES-GCM	128-256	AEAD	● Alta	Integridad + confidencialidad
AES-GCM-SIV	128-256	AEAD	● Alta	Resistente a reutilización de nonce
XChaCha20-Poly1305	256 bits	AEAD	● Muy Alta	Nonce de 192 bits, seguro para cifrado en reposo
AES-256-XTS	512 bits (2×256)	XTS	● Alta	Solo cifrado de disco (sin integridad)

⚠ **Modos INSEGUROS**: AES-ECB, AES-CBC (sin Encrypt-then-MAC), AES-CFB, AES-CTR (sin MAC)

⚠ Con GCM nunca se debe repetir el nonce con la misma clave.

Intercambio de Claves (Key Exchange)

Método	Tamaño	Seguridad	PFS	Recomendación
ECDHE-P256	256 bits	● Media	✓ Sí	Aceptable
ECDHE-P384	384 bits	● Alta	✓ Sí	Recomendado
ECDHE-P521	521 bits	● Muy Alta	✓ Sí	Máxima seguridad
X25519	256 bits	● Alta	✓ Sí	Moderno y rápido
X448	448 bits	● Muy Alta	✓ Sí	Mayor seguridad que X25519
DH grupos < 2048	1024 bits	● Baja	✗ No	Vulnerable (Logjam)
DH grupo 14	2048 bits	● Media	✓ Sí	Mínimo aceptable
DH grupo 19	256 bits (P-256)	● Alta	✓ Sí	ECDH moderno
DH grupo 21	521 bits (P-521)	● Muy Alta	✓ Sí	Máxima seguridad
X25519MLKEM768 (híbrido)	1216 bytes (clave pública)	● Muy Alta	✓ Sí	Recomendado, post-cuántico
RSA estático / DH-ECDH estático	—	● Baja	✗ No	✗ NO USAR (sin PFS; ROBOT, Marvin)

PFS = Perfect Forward Secrecy (si se compromete la clave privada, sesiones pasadas permanecen seguras)

Funciones Hash (Digest)

Hash	Tamaño	Colisiones	Seguridad	Uso
MD5	128 bits	✗ Encontradas	● NO USAR	Completamente roto
SHA-1	160 bits	✗ Prácticas	● NO USAR	Deprecado en HTTPS y firmas; HMAC-SHA1 solo obsoleto
SHA-256	256 bits	✓ Seguro	● Recomendado	Estándar actual
SHA-384	384 bits	✓ Seguro	● Recomendado	Mayor seguridad
SHA-512	512 bits	✓ Seguro	● Recomendado	Máxima seguridad
SHA-3	256/512	✓ Seguro	● Recomendado	Estándar moderno (FIPS 202)
BLAKE2	Variable	✓ Seguro	● Rápido	Más rápido que MD5

Contraseñas: no usar hashes simples (MD5, SHA-1, SHA-256) para almacenarlas; ver *Contraseñas (KDF) y cifrado en reposo*.

Firma Digital (Digital Signatures)

Algoritmo	Tamaño Clave	Seguridad	Velocidad	Recomendación
RSA-2048	2048 bits	🟠 Mínimo	Lenta	Aceptable hasta 2030 (NIST IR 8547); preferir ≥ 3072
RSA-3072	3072 bits	🟢 Alta	Lenta	Recomendado (≈128 bits)
RSA-4096	4096 bits	🟢 Alta	Lenta	Recomendado RSA
ECDSA-P256	256 bits	🟠 Media	Rápida	Aceptable
ECDSA-P384	384 bits	🟢 Alta	Rápida	Recomendado ECDSA
EdDSA (Ed25519)	256 bits	🟢 Muy Alta	Muy Rápida	✓ MEJOR OPCIÓN
EdDSA (Ed448)	456 bits	🟢 Extrema	Muy Rápida	Máxima seguridad
DSA	1024-3072 bits	🔴 Obsoleto	Lenta	✗ NO USAR (retirado en FIPS 186-5)

Ed25519 es la mejor opción moderna: segura, rápida y sin vulnerabilidades conocidas.

Para cifrar con RSA usar siempre RSA-OAEP, nunca PKCS#1 v1.5 (ROBOT, Marvin).

> **Nota**: RSA, ECDSA, EdDSA y ECDH son vulnerables a un ordenador cuántico. NIST IR 8547 (borrador) propone deprecarlos en 2030 y prohibirlos en 2035.

Protocolo SSH

SSH v2 (Recomendado)

Componente	Recomendado (Seguridad Media)	Recomendado (Seguridad Alta)
Host Key	ecdsa-sha2-nistp256, ssh-ed25519	ssh-ed25519
Key Exchange	curve25519-sha256	mlkem768x25519-sha256 (OpenSSH ≥ 9.9), curve25519-sha256
Encryption	aes256-gcm@openssh.com, aes256-ctr, chacha20-poly1305@openssh.com	chacha20-poly1305@openssh.com, aes256-gcm@openssh.com
MAC	hmac-sha2-256-etm@openssh.com	hmac-sha2-512-etm@openssh.com
Compression	Desactivada	Desactivada
Versión OpenSSH	≥ 10.6 (mínima recomendada; strict-KEX desde 9.6)	≥ 10.6 (mínima recomendada; ML-KEM híbrido desde 9.9)

```
# Configuración SSH segura (/etc/ssh/sshd_config)
# strict-KEX (Terrapin): OpenSSH ≥ 9.6 en cliente y servidor
# mlkem768x25519-sha256 requiere OpenSSH ≥ 9.9; con versiones anteriores, eliminarlo de KexAlgorithms
Protocol 2
HostKey /etc/ssh/ssh_host_ed25519_key
KexAlgorithms mlkem768x25519-sha256,curve25519-sha256,curve25519-sha256@libssh.org
Ciphers chacha20-poly1305@openssh.com,aes256-gcm@openssh.com
MACs hmac-sha2-512-etm@openssh.com,hmac-sha2-256-etm@openssh.com
```

Cómo interpretar las tablas de SSH

Las tablas de seguridad media y alta son **combinaciones recomendadas**, no una lista cerrada de lo único permitido. Se aplican con rigor cuando SSH se expone a Internet o a proveedores. En LAN internas y redes de gestión aisladas deberían ser el objetivo, aunque en muchos casos sea difícil de conseguir por las versiones de los equipos, del sistema operativo o de OpenSSH.

Nivel	Ejemplos
🟢 Recomendado seguridad alta (militar)	`mlkem768x25519-sha256`, `curve25519-sha256`, `chacha20-poly1305@openssh.com`, `aes256-gcm@openssh.com`, `hmac-sha2-256-etm@openssh.com`, `ssh-ed25519`
🟡 Aceptable por compatibilidad con lo que hay	`aes128-ctr`, `aes192-ctr`, `aes256-ctr`, `aes128-gcm@openssh.com`, `hmac-sha2-256/512` (sin `-etm`), `ecdh-sha2-nistp256/384/521`, `diffie-hellman-group14-sha256`, `group16-sha512`, claves `rsa-sha2-256/512` (RSA ≥ 3072), `ecdsa-sha2-nistp256`
🔴 Evitar: muy inseguros, rotos, vulnerables!	SSH v1, `3des-cbc`, `arcfour`, `blowfish-cbc`, `hmac-md5`, `hmac-sha1`, `diffie-hellman-group1-sha1`, `group14-sha1`, `ssh-rsa` (SHA-1), `ssh-dss`

- Los `aes*-cbc` están en el límite entre amarillo y rojo: solo si es imprescindible y con controles compensatorios.
- **SSH expuesto a Internet o a proveedores:** solo verde, o amarillo muy acotado y justificado, con OpenSSH parcheado, acceso por clave y filtrado por IP o VPN.
- **LAN internas y redes de gestión aisladas:** el verde es el objetivo. Se tolera amarillo mientras haya equipos que no puedan actualizarse, con controles compensatorios (segmentación, bastión o salto, registro y plan de actualización).
- **Versión mínima recomendada: OpenSSH 10.6.** Los mínimos técnicos de cada función son 9.6 (strict-KEX, Terrapin) y 9.9 (ML-KEM híbrido). En distribuciones con parches retroportados, comprobar el parche y no solo el número de versión.

IPsec (IKEv2)

Parámetro	Seguridad Media	Seguridad Alta
Protocolo	IKEv2	IKEv2 (no IKEv1)
Autenticación	SHA-256	SHA-512
Cifrado	AES-256-GCM	AES-256-GCM (o ChaCha20-Poly1305)
Grupo DH	Grupo 19 (P-256)	Grupo 20 (P-384) o 21 (P-521) + ML-KEM (RFC 9370, híbrido)
Integridad	SHA-256	SHA-512

WPA/WiFi

Estándar	Seguridad	Recomendación	Notas
WEP	🔴 Roto	✗ NO USAR	40-bit IV, completamente comprometido
WPA	🔴 Roto	✗ NO USAR	TKIP roto (Beck-Tews)
WPA2-Personal	🟠 Media	Aceptable	Solo con AES-CCMP (nunca TKIP), PMF y contraseña robusta; KRACK mitigado con parches
WPA2-Enterprise	🟢 Alta	Recomendado	Con 802.1X
WPA3-Personal	🟢 Alta	✓ Recomendado	SAE (Dragonblood mitigado en firmware actual); OWE es solo para redes abiertas (Enhanced Open)
WPA3-Enterprise	🟢 Muy Alta	✓ MEJOR	Modo 192 bits (GCMP-256, ECDH/ECDSA P-384, SHA-384)
WPA3 modo transición	🟠 Media	⚠ EVITAR	Permite downgrade a WPA2 (Dragonblood); exigir PMF

PGP/GPG

Componente	Recomendado (Media)	Recomendado (Alta)
Cifrado Simétrico	AES-192	AES-256
Cifrado Asimétrico	ECDH Curve25519	ECDH Curve25519 + X448
Firma	ECDSA P-256 o EdDSA	EdDSA (Ed25519)
Hash	SHA-256	SHA-512
Tamaño RSA (si se usa)	3072 bits	4096 bits
Formato / integridad	MDC obligatorio (EFAIL)	OpenPGP v6 (RFC 9580, AEAD OCB/GCM) si la implementación lo soporta

Contraseñas (KDF) y cifrado en reposo

Uso	Recomendado (Media)	Recomendado (Alta)	Evitar
Contraseñas	PBKDF2-HMAC-SHA256 (≥600.000 iteraciones), bcrypt (coste ≥12)	Argon2id, scrypt	MD5, SHA-1, SHA-256 sin sal ni KDF
Derivación de claves	HKDF-SHA256	HKDF-SHA384/512	Hash simple de la contraseña
Disco	AES-256-XTS	AES-256-XTS	AES-ECB, AES-CBC sin autenticar
Datos / ficheros	AES-256-GCM	AES-256-GCM-SIV, XChaCha20-Poly1305	Reutilizar nonce

Post-Cuántico (Quantum-Safe)

Algoritmo	Tipo	Tamaño Clave	Estado	Recomendación
ML-KEM-768 (Kyber)	Encapsulación	1184 bytes (clave pública)	🟢 NIST FIPS 203	Media/Alta
ML-KEM-1024	Encapsulación	1568 bytes (clave pública)	🟢 NIST FIPS 203	Muy Alta
ML-DSA (Dilithium)	Firma	1952 bytes (ML-DSA-65, clave pública)	🟢 NIST FIPS 204	Recomendado
SLH-DSA (SPHINCS+)	Firma	Variable	🟢 NIST FIPS 205	Stateless
FN-DSA (Falcon)	Firma	897 bytes (Falcon-512, clave pública)	🟠 FIPS 206 (borrador)	Compacto
X25519MLKEM768	KEM híbrido	1216 bytes	🟢 Desplegado en navegadores y CDN	Recomendado (TLS 1.3)
HQC	Encapsulación (códigos)	—	🟠 Seleccionado por NIST (2025), en estandarización	Respaldo de ML-KEM
LMS / XMSS	Firma con estado	—	🟢 NIST SP 800-208	Firmware y arranque seguro

Nota: Los algoritmos post-cuánticos de NIST (FIPS 203, 204 y 205, agosto de 2024) son los más actuales y recomendados. Se aconseja el modo **híbrido** (clásico + post-cuántico).

Cifrados y protocolos no recomendados

Matriz de cifrados INSEGUROS

Categoría	Algoritmos	Vulnerabilidades	Riesgo
Cifrados de Bloque Débiles	DES, 3DES, RC2, IDEA, CAST5, Blowfish	Clave o bloque pequeño (Sweet32)	🔴 CRÍTICO
Cifrados Stream	RC4	Sesgos en el keystream (Bar Mitzvah, RC4 NOMORE)	🔴 CRÍTICO
Cifrados Modernos (Mal Usados)	AES-ECB, AES-CBC (sin EtM), AES-CFB, AES-CTR sin MAC	Patrones (ECB), bit-flipping, padding oracle (CBC)	🔴 CRÍTICO
Funciones Hash	MD2, MD4, MD5, SHA-1	Colisiones prácticas	🔴 CRÍTICO
Intercambio Claves	Diffie-Hellman < 2048	Logjam, pre-computation	🔴 CRÍTICO
Cifrado / intercambio RSA	RSA estático, RSA PKCS#1 v1.5	ROBOT, Marvin (padding oracle), sin PFS	🔴 CRÍTICO
Suites obsoletas	NULL, EXPORT, anónimas (aNULL/eNULL)	FREAK, Logjam	🔴 CRÍTICO

No rotos, pero no recomendados (compatibilidad): CAMELLIA, ARIA, Twofish y Serpent no tienen ataques prácticos, pero están poco soportados; preferir AES o ChaCha20.

Vulnerabilidades por protocolo/algoritmo

Vulnerabilidad	Afecta a	Año	CVSS	Mitigación
POODLE	SSLv3, TLS 1.0	2014	7.1	Deshabilitar SSLv3
Heartbleed	OpenSSL 1.0.1 a 1.0.1f	2014	7.5	Actualizar OpenSSL
FREAK	OpenSSL, exportación	2015	5.9	Deshabilitar suites EXPORT y actualizar
Logjam	DH < 2048 bits	2015	5.9	Usar DH >= 2048 o ECDHE
DROWN	SSLv2	2016	7.5	Deshabilitar SSLv2
Sweet32	3DES, Blowfish	2016	6.8	No usar cifrados de bloque de 64 bits
KRACK	WPA2 (4-way handshake)	2017	8.1	Parchear clientes y AP; usar WPA3
BEAST	AES-CBC en TLS 1.0	2011	5.9	Usar TLS 1.2+
Lucky13	CBC mode en TLS	2013	5.1	Usar AEAD (GCM)
CRIME	Compresión TLS	2012	7.5	Deshabilitar compresión
SLOTH	TLS 1.2 con MD5/SHA-1 en firmas	2016	—	No usar MD5/SHA-1 en firmas
SHAttered	SHA-1	2017	—	Migrar a SHA-256 o superior
ROBOT	RSA PKCS#1 v1.5 en TLS	2017	—	Deshabilitar RSA estático
EFAIL	PGP / S-MIME (CBC/CFB sin MDC)	2018	—	Exigir MDC/AEAD y actualizar clientes
Dragonblood	WPA3-SAE	2019	—	Actualizar firmware; evitar modo transición
Raccoon	DH en TLS ≤ 1.2	2020	—	Usar ECDHE o TLS 1.3
FragAttacks	WiFi (WEP a WPA3)	2021	—	Parchear dispositivos
Terrapin	SSH (ChaCha20-Poly1305, CBC-EtM)	2023	5.9 (CVE-2023-48795)	OpenSSH ≥ 9.6 con strict-KEX
Marvin	RSA PKCS#1 v1.5 (descifrado)	2023	—	Usar RSA-OAEP o ECDHE
BlastRADIUS	RADIUS (MD5)	2024	—	Actualizar RADIUS; usar Message-Authenticator o RADIUS sobre TLS

Protocolos específicos a evitar

```
SSH v1:
  - ✗ Completamente roto
  - Usar: SSH v2 siempre

Cifrados SSH a evitar:
  - 3des-cbc, arcfour, blowfish-cbc, aes*-cbc
  - hmac-md5, hmac-sha1 (sin -etm)
  - OpenSSH < 9.6 sin strict-KEX (Terrapin)

IKEv1:
  - ✗ Obsoleto (incluido el modo agresivo)
  - Usar: IKEv2 siempre

Grupos DH:
  - ✗ Grupo 1 (768 bits) - Totalmente roto
  - ✗ Grupo 2 (1024 bits) - Vulnerable a Logjam
  - ✗ Grupo 5 (1536 bits) - Insuficiente
  - ✓ Mínimo: Grupo 14 (2048 bits)

WEP/WPA:
  - ✗ Completamente comprometido
  - ✗ WPA2 con TKIP, WPS activado
  - Usar: WPA2-AES o WPA3 siempre

TLS 1.2 mal configurado:
  - ✗ Suites CBC, RSA estático, sin AEAD
  - Usar: ECDHE + AEAD (GCM o ChaCha20)
```

Certificados digitales

Certificados Digitales

Recomendaciones

Aspecto	Recomendación	Por qué
Autoridad Certificadora	CA reconocida (Let's Encrypt, Sectigo, etc.)	Validación de confianza
Algoritmo Firma	SHA-256, SHA-384, SHA-512	SHA-1 está deprecado
Algoritmo Clave	RSA-2048+ (mejor ≥ 3072), ECDSA-P256/P384; EdDSA solo en PKI privada	Resistencia y compatibilidad con CA públicas
Tipo Certificado	DV, OV, EV (según necesidad)	Validación de dominio/organización
Validez	Máx. 200 días (desde mar-2026), 100 días (2027) y 47 días (2029) según CA/B Forum; automatizar con ACME	Rotación frecuente = seguridad
SAN	Múltiples dominios si es necesario	Flexibilidad y costo
OCSP Stapling	Habilitado	Revocación sin latencia
Certificate Transparency	Monitorizar los logs CT de tus dominios	Detecta certificados emitidos sin autorización
ECH	Encrypted Client Hello donde sea posible	Oculta el SNI

Herramientas para Verificar Certificados

```
# Ver detalles del certificado
openssl x509 -in cert.pem -text -noout

# Verificar cadena de certificados
openssl verify -CAfile ca.pem cert.pem

# Verificar con OpenSSL s_client
openssl s_client -connect example.com:443 -showcerts
```

Herramientas de Escaneo

Este repositorio incluye herramientas automáticas para auditar la seguridad criptográfica:

testssl.sh

Descripción: Auditoría completa de TLS/SSL en servidores web

Uso:

```
./testssl.sh https://ejemplo.com
./testssl.sh 192.168.1.1:443
./testssl.sh --full ejemplo.com
./testssl.sh --jsonfile results.json ejemplo.com
```

Funcionalidades:

- Detecta versión TLS/SSL
- Enumera cifrados soportados
- Identifica vulnerabilidades conocidas
- Genera reportes en JSON/HTML
- Verifica certificados
- Pruebas de OCSP, CRL
- Análisis de seguridad PFS

Requisitos: bash, openssl, curl

scan_cifrados.sh

Descripción: Escanea múltiples direcciones IP y clasifica por nivel de seguridad

Uso:

```
sh scan_cifrados.sh
# Lee IPs desde: ip.txt (una por línea)

# Archivo ip.txt:
192.168.1.1:443
10.0.0.1:22
ejemplo.com:443
```

Salida:

- Clasifica en: `seguridad_alta.txt`, `seguridad_media.txt`, `seguridad_baja.txt`, `seguridad_militar.txt`
- Genera reportes por nivel de seguridad

filtro.sh

Descripción: Filtra y procesa resultados de escaneos

Uso:

```
./filtro.sh <archivo_entrada> <filtro>
# Ejemplo:
./filtro.sh resultados.txt "AES-256"
```

ssh-audit

Descripción: Auditoría de servidores y clientes SSH (algoritmos, claves, Terrapin)

Uso:

```
ssh-audit ejemplo.com
ssh-audit -p 2222 192.168.1.1
```

nmap (ssl-enum-ciphers)

Descripción: Enumera protocolos y cifrados TLS soportados y los califica

Uso:

```
nmap --script ssl-enum-ciphers -p 443 ejemplo.com
```

Guía de Instalación

Requisitos Previos

```
# Ubuntu/Debian
sudo apt-get update
sudo apt-get install -y openssl curl bash git nmap

# CentOS/RHEL
sudo yum install -y openssl curl bash git nmap

# macOS
brew install openssl curl bash nmap

# ssh-audit (todas las plataformas)
pip install ssh-audit
```

Descarga del Repositorio

```
git clone https://github.com/hackingyseguridad/cifrados.git
cd cifrados
chmod +x testssl.sh scan_cifrados.sh filtro.sh
```

Configuración Inicial

```
# Crear archivo de IPs para escanear
cat > ip.txt << EOF
ejemplo.com:443
192.168.1.1:443
10.0.0.1:22
EOF

# Ejecutar escaneo
sh scan_cifrados.sh

# Ver resultados
cat seguridad_alta.txt
cat seguridad_media.txt
cat seguridad_baja.txt
```

Casos de Uso

securizar servidor Web (Apache/Nginx)

```
# 1. Auditar servidor actual
./testssl.sh https://tu-servidor.com

# 2. Implementar configuración segura
# En Apache: /etc/apache2/mods-enabled/ssl.conf
SSLProtocol -all +TLSv1.3 +TLSv1.2
SSLCipherSuite
    ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CH
    ACHA20-POLY1305
SSLOpenSSLConfCmd Curves X25519:X448:secp384r1
# Con OpenSSL ≥ 3.5 (híbrido post-cuántico):
# SSLOpenSSLConfCmd Curves X25519MLKEM768:X25519:secp384r1

# En Nginx: /etc/nginx/nginx.conf
ssl_protocols TLSv1.3 TLSv1.2;
ssl_ciphers ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDH
    E-RSA-CHACHA20-POLY1305;
ssl_ecdh_curve X25519:X448:secp384r1;
# Con OpenSSL ≥ 3.5 (híbrido post-cuántico):
# ssl_ecdh_curve X25519MLKEM768:X25519:secp384r1;
ssl_prefer_server_ciphers on;
```

Configurar SSH Seguro

```
# Editar /etc/ssh/sshd_config
# (mlkem768x25519-sha256 requiere OpenSSH ≥ 9.9; con versiones anteriores, eliminarlo)
Protocol 2
HostKey /etc/ssh/ssh_host_ed25519_key
KexAlgorithms mlkem768x25519-sha256,curve25519-sha256,curve25519-sha256@libssh.org
Ciphers chacha20-poly1305@openssh.com,aes256-gcm@openssh.com
MACs hmac-sha2-512-etm@openssh.com,hmac-sha2-256-etm@openssh.com

# Reiniciar SSH
sudo systemctl restart sshd

# Verificar configuración
ssh-audit tu-servidor.com
```

auditoría de seguridad

```
# Crear lista de servidores
echo "servidor1.empresa.com" > servidores.txt
echo "servidor2.empresa.com" >> servidores.txt
echo "db.empresa.com:3306" >> servidores.txt

# Escanear todos
for server in $(cat servidores.txt); do
    echo "Escaneando $server..."
    ./testssl.sh --jsonfile "$server.json" "$server"
done

# Analizar resultados
grep -lE '"severity" *: *"(HIGH|CRITICAL)"' *.json > fallos.txt
```

migración a post-cuántico

```
# 1. Auditar algoritmos actuales
./testssl.sh tu-servidor.com | grep "Cipher"

# 2. Planificar migración
# - Fase 1: TLS 1.3 + ECDHE moderno
# - Fase 2: Híbrido X25519MLKEM768 (X25519 + ML-KEM-768)
# - Fase 3: ML-KEM-1024 + ML-DSA

# 3. Implementar con OpenSSL ≥ 3.5 y OpenSSH ≥ 9.9 (soporte ML-KEM nativo)
openssl list -kem-algorithms | grep -i ml-kem
ssh -Q kex | grep mlkem
```

Resumen

Por Tipo de Servicio

Servicio	Protocolo	Cifrado	Key Exchange	Hash	Firma
HTTPS	TLS 1.3	AES-256-GCM	X25519MLKEM768 (híbrido)	SHA-256	ECDSA-P384
SSH	SSH v2	ChaCha20-Poly1305	mlkem768x25519-sha256	SHA-256	Ed25519
IPsec	IKEv2	AES-256-GCM	DH-21 (P-521)	SHA-512	EdDSA
VPN	WireGuard	ChaCha20-Poly1305	X25519	BLAKE2	Ed25519
WiFi	WPA3	AES-256-GCMP	SAE	SHA-384	N/A
Base de Datos	TLS 1.3	AES-256-GCM	ECDHE-P384	SHA-256	Ed25519
Almacenamiento	AES-256	AES-256 (XTS)	Derivación clave	SHA-512	N/A
Correo	TLS 1.3	AES-256-GCM	ECDHE-P256	SHA-256	EdDSA

Por Nivel de Seguridad

- SEGURIDAD ALTA (Recomendado para 2025-2030):
  - TLS 1.3 / SSH v2 / IKEv2
  - AES-256-GCM o ChaCha20-Poly1305
  - ECDHE con curvas modernas (P-384, X25519, X448)
  - Híbrido post-cuántico X25519MLKEM768 donde esté disponible
  - SHA-256/384/512
  - EdDSA (Ed25519) para firmas

- SEGURIDAD MEDIA (Aceptable con cuidado):
  - TLS 1.2 con cifrados GCM
  - AES-128-GCM (no recomendado solo)
  - ECDHE-P256 mínimo
  - SHA-256 mínimo
  - ECDSA-P256 o RSA-3072+

- SEGURIDAD BAJA (NO USAR):
  - SSLv3, TLS 1.0, TLS 1.1
  - DES, 3DES, RC2, RC4, IDEA, CAST5, Blowfish
  - Suites NULL / EXPORT / anónimas
  - RSA estático (sin PFS)
  - MD5, SHA-1
  - DH < 2048 bits
  - RSA < 2048 bits

- SEGURIDAD MILITAR (referencia: CNSA 2.0):
  - AES-256, SHA-384/512
  - ECDH/ECDSA P-384
  - ML-KEM-1024, ML-DSA-87
  - LMS/XMSS para firmware

Estructura del repositorio

```
cifrados/
├── README.md                    # Resumen: cifrados recomendados y no recomendados
├── LEEME.md                     # Este archivo (guía detallada)
├── CIFRADOS.md                  # Guía detallada de cifrados
├── AES-128.md                   # Ejemplos AES-128
├── SKILL.md
├── testssl.sh                   # Auditoría de TLS/SSL
├── scan_cifrados.sh             # Escaneo múltiples servidores
├── filtro.sh                    # Procesar resultados
├── cifrarfichero.sh             # Cifrar un fichero
├── seguridad_alta.txt           # Resultado: servidores seguros
├── seguridad_media.txt          # Resultado: seguridad media
├── seguridad_baja.txt           # Resultado: servidores inseguros
└── seguridad_militar.txt        # Resultado: máxima seguridad
```

Referencias

Estándares Oficiales

- NIST Special Publication 800-175B - Guía de Criptografía
- RFC 8446 - TLS 1.3
- RFC 8308 - Extension Negotiation en SSH
- RFC 8439 - ChaCha20-Poly1305 (obsoleta RFC 7539)
- RFC 8996 - Deprecación de TLS 1.0 y 1.1
- RFC 9180 - HPKE
- RFC 9370 - Múltiples intercambios de claves en IKEv2
- RFC 9580 - OpenPGP

Documentación de Seguridad

- OWASP - Cryptographic Failures
- Mozilla SSL Configuration Generator
- SANS Top 25 Weaknesses
- NIST PQC Project
- FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA)
- NIST IR 8547 - Transición a criptografía post-cuántica
- Terrapin Attack

Herramientas Relacionadas

- testssl.sh - Auditoría TLS
- ssh-audit - Auditoría SSH
- OpenSSL - Herramienta estándar
- Wireshark - Análisis de tráfico

Artículos Técnicos

- Criptografía Post-Cuántica
- Vulnerabilidades TLS Históricas
- Migración de Algoritmos Legacy
- Gestión de Certificados

Disclaimer

Este repositorio proporciona información sobre seguridad criptográfica. El uso de esta información es bajo tu responsabilidad. No nos hacemos responsables de:

- Daños causados por configuraciones incorrectas

- Pérdida de datos o sistemas
- Uso malintencionado de esta información
- Cambios en estándares de seguridad después de la publicación

http://www.hackingyseguridad.com/

Fuente original y descarga

Este documento reúne en PDF el contenido del repositorio **hackingyseguridad/cifrados**, que es su **fuente original** y el **lugar de descarga** de la versión más reciente de este PDF (`cifradosrecomendados.pdf`):

https://github.com/hackingyseguridad/cifrados/

En el repositorio encontrarás además **mayor detalle e información sobre cifrados recomendados**:

- README.md: resumen de cifrados recomendados y no recomendados (Parte I de este documento).
- LEEME.md: guía detallada (Parte II de este documento).
- CIFRADOS.md: segunda guía de cifrados recomendados.
- `testssl.sh`, `scan_cifrados.sh` y `filtro.sh`: herramientas de escaneo y clasificación por nivel de seguridad.

Sitio web: http://www.hackingyseguridad.com/

Elaborado con apoyo de Inteligencia artificial y revisado por el autor.